SIMATS ENGINEERING

Engineer to Excel

SIMATS

Saveetha Institute of Medical And Technical Sciences

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 Data Warehousing and Data Mining
Assignment 3 – CO3

Course Outcome: CO3 – Understand the functionality of the various data mining components and expose them to real-world data sets to analyse their behaviour.

Name: Harini S

Register Number: 192511153

Department: Computer science and engineering

Subject: CSA1601 – Data Warehousing and Data Mining

Academic Year: 2026–2027

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSA16 - DATA WAREHOUSING AND DATA MINING

ASSIGNMENT 3 - CO3

Course Outcome: CO3 - Understand the functionality of the various data mining components and expose them to real-world data sets to analyse their behaviour.

CO3 Coverage: CLASSIFICATION AND PREDICTION: Issues regarding classification and prediction, decision tree induction, Bayesian classification, rule-based classification, backpropagation, support vector machines, associative classification, accuracy and error measures, evaluating a classifier or predictor (cross-validation and bootstrap), metrics for performance evaluation, prediction algorithms, and the Weka tool for mining.

Mode: Pair Work - 2 Students per Group | Duration: 1 Week | Total Marks: 50

Pair-Work Principle

Both students are responsible for understanding the complete assignment. However, each question contains clearly identified individual responsibilities so that the contribution and understanding of each student can be independently verified.

Student A and Student B must not divide the assignment question-wise. Both students must participate in every question.

The individual responsibilities intentionally alternate across questions so that both students demonstrate evidence of:

Calculate to Implement to Analyse to Evaluate

Q1 - Decision Tree Induction: Choosing the Splitting Attribute (6 Marks)

A bank records the following loan decisions:

#	Income	Credit History	Employment	Approved
1	High	Good	Salaried	Yes
2	High	Poor	Salaried	No
3	High	Good	Self-employed	Yes
4	Medium	Good	Salaried	Yes
5	Medium	Poor	Self-employed	No
6	Medium	Good	Self-employed	No
7	Medium	Poor	Salaried	No
8	Low	Good	Salaried	Yes
9	Low	Poor	Self-employed	No
10	Low	Poor	Salaried	No
11	Low	Good	Self-employed	No
12	High	Poor	Self-employed	Yes

Shared Group Work

Together:

- Identify the class attribute, the candidate splitting attributes and the stopping conditions
- Agree on the splitting criterion to be used for the manual tree and on the pruning approach
- Install Weka and load the same data (ARFF/CSV) for the experiments

Student A - Manual Induction

- Compute the entropy of the full data set and the information gain of every attribute at the root
- Compute the gain ratio or the Gini index for any one attribute and state whether it changes the choice of the root
- Build the complete tree and extract it as a set of paths
- Explain what happens if an attribute has many distinct values (for example a customer ID)

Student B - Weka Validation

- Run J48 on the 12 tuples, and then on the German Credit (credit-g) data set that ships with Weka
- Compare Student A's manual tree with the Weka tree and explain every difference

- Vary the pruning confidence factor and minimum instances per leaf; report tree size and accuracy
- Explain overfitting using your own evidence

Individual Verification

Each student must be able to explain the other student's part of Q1.

Evidence Expected

Manual gain computations, the tree, the Weka output with parameter settings, and an interpreted comparison.

Expanded explanation: Decision tree induction is a supervised classification method in which the training records are repeatedly divided into smaller groups according to the values of selected attributes. In this question, Approved is the class attribute because it is the outcome to be predicted. Income, Credit History and Employment are the candidate splitting attributes. A good split produces child groups that contain records belonging mainly to one class.

Entropy is used to measure the impurity of the class labels. It is calculated as H(S) = -sum pi log2(pi), where pi is the proportion of records belonging to class i. The loan data contains 5 approved and 7 rejected applications, so the root entropy is approximately 0.979 bits. Information gain is the difference between the root entropy and the weighted entropy after splitting on an attribute. The attribute with the highest information gain is selected as the root under the ID3 criterion.

For the given data, the root information gains are approximately: Income = 0.073, Credit History = 0.203 and Employment = 0.003 bits. Therefore, Credit History gives the greatest reduction in entropy and is selected as the root using information gain. Gain ratio divides information gain by split information, while Gini impurity measures the probability of incorrectly classifying a randomly selected record. These criteria may rank attributes differently, so the selected criterion must be stated clearly.

The tree is expanded recursively for each branch until the records in a node belong to one class, no useful attribute remains, or a stopping rule is reached. If no further split is possible, the majority class can be assigned to the leaf. The final tree can be converted into IF-THEN paths by writing the conditions from the root to each leaf. A customer ID with a unique value for every row can create many tiny branches and memorise the training records. This can cause overfitting and poor performance on unseen applicants.

J48 in Weka is an implementation of the C4.5 decision-tree approach. Its output should be compared with the manually calculated tree, taking account of the splitting criterion, pruning, minimum leaf size and tie-breaking. When the confidence factor is reduced, pruning generally becomes more aggressive; increasing the minimum number of instances per leaf restricts small branches. Record the actual tree size and accuracy from each Weka run rather than assuming a particular result.

Q2 - Bayesian Classification and the Cost of Assumptions (6 Marks)

Shared Group Work

Using the loan data of Q1, classify the unseen applicant X = (Income = Low, Credit History = Good, Employment = Self-employed) using the Naïve Bayes classifier.

Student A - Calculation

- Compute the prior probabilities and all required conditional probabilities
- Find the posterior (proportional) value for each class and the predicted class
- Show what happens if one conditional probability is zero, and fix it with Laplacian correction; recompute

Student B - Investigation

- Run NaiveBayes in Weka on the same data and verify Student A's probabilities
- Create a data set (real or synthetic) in which two attributes are strongly correlated, and measure how the accuracy of Naïve Bayes changes compared with a decision tree
- Explain why Naïve Bayes can still give useful class rankings although the independence assumption is violated

Joint Conclusion

- When would you prefer Naïve Bayes over a decision tree, and when would you not?

Evidence Expected

- Probability tables, posterior calculations, Laplace correction, Weka verification and evidence for the independence-assumption study.

Expanded explanation: Naïve Bayes applies Bayes' theorem to estimate the probability that an applicant belongs to a class after observing the applicant's attribute values. The two possible classes in the loan data are Approved = Yes and Approved = No. The prior probabilities are calculated from the class frequencies: P(Yes) = 5/12 and P(No) = 7/12.

For applicant X = (Low, Good, Self-employed), the required conditional probabilities are estimated by counting each attribute value within each class. For Approved = Yes, P(Low|Yes) = 1/5, P(Good|Yes) = 4/5 and P(Self-employed|Yes) = 2/5. For Approved = No, P(Low|No) = 3/7, P(Good|No) = 2/7 and P(Self-employed|No) = 3/7. These values are multiplied by the corresponding prior to obtain proportional posterior scores.

The proportional score for Yes is (5/12) x (1/5) x (4/5) x (2/5) approximately 0.0267. The proportional score for No is (7/12) x (3/7) x (2/7) x (3/7) approximately 0.0306. Since the score for No is slightly higher, the Naïve Bayes prediction without smoothing is Approved = No. The scores can be normalised by dividing each by their sum to obtain posterior probabilities that add to one.

Naïve Bayes assumes that the attributes are conditionally independent given the class. If two attributes are strongly correlated, their evidence may effectively be counted more than once. Laplace correction addresses zero-frequency estimates by adding one to each possible category count. It prevents a zero conditional probability from forcing the complete class score to zero. Student B should compare the Weka probability output with the manual calculations and document the precise data representation and settings.

Q3 - Rule-Based and Associative Classification (7 Marks)

Shared Group Work

- Together, convert the Q1 decision tree into IF-THEN rules and discuss how rules represent knowledge differently from a tree.

Student A - Rule Quality

- Extract the rules and compute coverage and accuracy of each rule on the 12 tuples
- Explain rule conflict resolution (size ordering, rule ordering) with one example from the data
- Explain the sequential-covering idea for learning rules one at a time, and what is removed after each rule is learned

Student B - Associative Classification

- Run JRip (or PART) in Weka on credit-g and compare the rules with those obtained from J48
- Explain how associative classification (class-association rules) differs from the rules above, and what the roles of support and confidence are
- Mine class-association rules from the 12-tuple data with minimum support 25% and confidence 70%, and list those predicting Approved = Yes

Joint Design Question

- A loan officer must be able to explain every rejection to the customer. Which of the three representations (tree, rules, associative rules) would you recommend, and why?

Evidence Expected

- Rule list with coverage/accuracy, JRip/PART output, class-association rules and a justified recommendation.

Expanded explanation: A decision tree can be represented as a collection of IF-THEN rules. Each path from the root node to a leaf becomes one rule. The conditions along the path form the IF part, and the class at the leaf forms the THEN part. This representation allows a loan officer to inspect the conditions that led to a particular prediction without tracing the entire tree.

Rule coverage is the number of records whose attribute values satisfy the rule conditions. Rule accuracy is the proportion of those covered records that belong to the class predicted by the rule. Both measures should be reported because a rule may be very accurate but apply to only a small number of applicants. A rule with broad coverage may be useful operationally but can also include more exceptions.

Rules can overlap, meaning that one record may satisfy more than one rule. If the rules predict different classes, a conflict-resolution method is required. Common approaches include preferring the more specific rule, following a fixed rule order, or using confidence or support to select a prediction. Sequential covering learns one rule at a time, removes or marks the positive examples covered by that rule, and continues until a stopping condition is met.

Associative classification creates class-association rules from frequent itemsets. Support measures the proportion of the full data set containing both the rule antecedent and the predicted class. Confidence measures the proportion of records satisfying the antecedent that also belong to the predicted class. With minimum support 25% on 12 records, a rule must occur in at least 3 records; with minimum confidence 70%, at least 70% of the records matching its condition must have the predicted class. The discovered rules should be listed with their support and confidence.

Q4 - Backpropagation: What Does the Network Learn? (7 Marks)

Shared Group Work

- Consider a network with 2 inputs, 2 hidden units and 1 output unit using the sigmoid activation. Inputs x1 = 1, x2 = 0, target t = 1, learning rate 0.9. Initial weights: w(x1 to h1) = 0.2, w(x2 to h1) = 0.4, w(x1 to h2) = -0.3, w(x2 to h2) = 0.1, w(h1 to o) = -0.3, w(h2 to o) = -0.2; biases: h1 = -0.4, h2 = 0.2, o = 0.1.

Student A - One Training Step

- Compute the forward pass: output of each hidden unit and the output unit
- Compute the error terms of the output and hidden units
- Update all weights and biases for one iteration and recompute the output; did the error decrease?
- Explain why the weights are initialised to small random values and what happens if all are set to the same value

Student B - Behaviour of the Learner

- Train a MultilayerPerceptron in Weka (or equivalent in Python) on a real data set
- Vary the learning rate, the number of hidden units and the number of epochs; record training and test accuracy
- Identify and demonstrate overfitting and slow/unstable training
- Compare training time and interpretability with the decision tree of Q1

Individual Verification

- Each student must be able to explain how the output error propagates back to a hidden-layer weight.

Evidence Expected

- Hand calculations for one iteration, tables of experiments and a discussion of strengths and limitations of neural classifiers.

Expanded explanation: Backpropagation is the learning procedure used to adjust the weights of a multilayer neural network so that its predicted output moves closer to the target. The network in this question has two input units, two hidden units and one output unit. Each neuron calculates a weighted sum of its incoming values and bias, then applies the sigmoid activation function sigmoid(z) = 1/(1 + e-z).

During the forward pass, the input values are multiplied by their connection weights and added to the relevant bias. The resulting net input is passed through the sigmoid function to obtain each hidden-unit output. The hidden outputs are then passed to the output unit in the same way. The difference between the target value and predicted output represents the prediction error.

For a sigmoid output unit, the error term is commonly calculated as deltao = (t - o)o(1 - o), where t is the target and o is the output. The hidden-unit error term depends on the hidden activation derivative and the weighted error propagated back from the output unit. Each weight is updated using the learning rate multiplied by the relevant input activation and error term. Biases are updated using the corresponding error term.

After updating the parameters, perform the forward pass again and compare the new squared error with the previous error. A reduction indicates that this training step moved the output in the desired direction. Small random initial weights help prevent all hidden units from learning identical features. If all weights are initialised identically, symmetry may cause hidden units to receive identical updates and behave as copies of one another.

In the Weka MultilayerPerceptron experiment, change one parameter at a time where possible. Record learning rate, number of hidden units, training time, epochs, training accuracy and test accuracy. A large learning rate can make updates unstable, while a very small rate can slow convergence. A large network or excessive training can overfit, so test performance must be considered alongside training performance.

Q5 - Support Vector Machines: Margin, Kernels and Scaling (7 Marks)

Shared Group Work

- Six points in 2-D: Class +1 to (3,3), (4,3), (4,4); Class -1 to (1,1), (1,2), (2,1). Together, plot them and discuss what a maximum-margin separator means.

Student A - Geometry

- Identify by inspection the support vectors, and write a separating hyperplane w*x + b = 0 that is equidistant from the two classes
- Compute the margin width for your hyperplane
- Explain what changes if a non-support-vector point is removed, and what changes if a support vector is removed
- Explain the role of the soft margin parameter C and of kernels for non-linearly separable data

Student B - Experiments

- Using SMO in Weka (or SVC in Python), classify a non-linear data set (for example concentric circles or a real data set) with linear, polynomial and RBF kernels
- Study the effect of C and kernel parameters on accuracy and the number of support vectors
- Show the effect of not scaling the attributes and then of scaling them

Joint Conclusion

- For which type of data would you choose an SVM, and what practical limitations did your experiments reveal?

Evidence Expected

- Plot with support vectors, hyperplane and margin, kernel/parameter experiments and the effect of scaling.

Expanded explanation: A support vector machine is a supervised learning algorithm that attempts to find a decision boundary separating classes with the largest possible margin. The margin is the distance between the separating hyperplane and the nearest training points from either class. These nearest points are called support vectors because they determine the location of the boundary.

For a linear SVM, the separating hyperplane is written as $w^*x + b = 0$. The two margin boundaries are $w^*x + b = +1$ and $w^*x + b = -1$, and the margin width is $2/||w||$. For the six points in the question, the two classes are separated diagonally; the closest points to the boundary determine the margin. The selected w and b should be checked by substituting the support-vector coordinates into the boundary equations.

If a point that is not a support vector is removed, the optimal separating hyperplane usually remains unchanged because that point does not determine the margin. Removing a support vector can change the boundary and margin because the optimisation is directly influenced by those points. This explains why support vectors are central to the SVM solution.

The parameter C controls the trade-off between a wide margin and training errors. A large C penalises margin violations strongly, while a smaller C allows more violations and may produce a wider-margin solution. Kernel functions allow an SVM to model non-linear decision boundaries. A linear kernel is suitable when a straight boundary is adequate; polynomial and RBF kernels can represent more complex patterns.

Feature scaling is important because SVM calculations depend on distances and dot products. If one feature has values in the thousands while another ranges from zero to one, the large-scale feature may dominate the model. Compare results before and after scaling, and report the kernel, C value, kernel parameters, accuracy and support-vector count for each experiment.

Q6 - Evaluating Classifiers and Predictors (7 Marks)

Shared Group Work

- A fraud-detection classifier is tested on 1,000 transactions and produces: TP = 40, FN = 20, FP = 30, TN = 910. Together, discuss which measure is meaningful for this problem.

Student A - Metrics and Prediction Error

- Compute accuracy, error rate, sensitivity (recall), specificity, precision and F-measure
- Show that a classifier that predicts "not fraud" for every transaction obtains a high accuracy, and explain the class-imbalance problem
- For actual values {12, 15, 20, 25} and predicted values {10, 18, 19, 30}, compute MAE, MSE, RMSE of a predictor and explain when RMSE is preferred over MAE
- Explain how a ROC curve is constructed and how to read it

Student B - Estimation Methods

- On an imbalanced data set (for example credit-g or a fraud data set), compare holdout, 10-fold cross-validation and bootstrap (.632) estimates of accuracy
- Repeat holdout with at least five random splits and report the variation in accuracy
- Compare J48, Naïve Bayes and SMO using paired evaluation (for example Weka Experimenter) and report whether the differences are meaningful
- Plot a ROC curve for one classifier

Individual Verification

- Each student must be able to explain why accuracy alone is misleading and which estimation method they would trust and why.

Evidence Expected

- Metric calculations, error measures, estimation-method comparison with variation and an interpreted classifier comparison.

Expanded explanation: A confusion matrix compares predicted classes with actual classes. For this fraud-detection example, TP = 40, FN = 20, FP = 30 and TN = 910, giving 60 actual fraud cases and 940 non-fraud cases. The total number of transactions is 1,000. These counts allow several measures to be calculated and interpreted.

Accuracy = (TP + TN)/Total = (40 + 910)/1000 = 95%. Error rate = (FP + FN)/Total = 50/1000 = 5%. Sensitivity or recall = TP/(TP + FN) = 40/60 = 66.67%. Specificity = TN/(TN + FP) = 910/940 approximately 96.81%. Precision = TP/(TP + FP) = 40/70 approximately 57.14%. The F-measure, calculated as the harmonic mean of precision and recall, is approximately 61.54%.

A classifier that predicts every transaction as not fraud would be correct for all 940 non-fraud transactions and wrong for all 60 fraud transactions. Its accuracy would still be 94%, even though it detects no fraud at all. This demonstrates why accuracy alone is insufficient for imbalanced data. Recall is important when missing fraud is costly, while precision is important when false alerts create significant investigation effort.

For the prediction values in the question, the absolute errors are 2, 3, 1 and 5. Therefore MAE = (2 + 3 + 1 + 5)/4 = 2.75. The squared errors are 4, 9, 1 and 25, so MSE = 39/4 = 9.75 and RMSE = sqrt9.75 approximately 3.12. RMSE penalises larger errors more heavily than MAE and is useful when large prediction mistakes should receive greater weight.

A ROC curve is created by varying the classification threshold and plotting the true-positive rate against the false-positive rate at each threshold. A point closer to the top-left corner indicates a favourable balance between detecting positives and limiting false positives. Cross-validation, holdout and bootstrap are estimation approaches; repeated splits help show how much performance changes with the selected training and test samples. Actual experimental comparisons must be reported from the chosen data set and software.

Q7 - Design a Classification Solution from First Principles (10 Marks)

Problem

A hospital wants to identify patients likely to be readmitted within 30 days. Only 6% of patients are readmitted. Missing a high-risk patient (false negative) is judged ten times more costly than flagging a patient who is not at risk (false positive). Doctors must be able to understand the reason for each alert. Data: age, diagnosis group, length of stay, number of prior admissions, lab results and discharge medications (some values missing).

Stage 1 - Independent Design Proposal

Before discussing the solution with each other, Student A and Student B each independently propose:

- The classification technique(s) they would use
- Preprocessing and handling of imbalance and missing values
- The evaluation measure and estimation method
- How the model's decisions will be explained to doctors
- Each student's initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect. The purpose is to provide evidence of independent thinking.

Stage 2 - Compare the Two Designs

- Discuss where the designs agree and differ, the strengths and weaknesses of each, missing cases, and which ideas should be retained. Record this comparison.

Stage 3 - Joint Final Design

- Develop the final solution, identifying the chosen technique, preprocessing, parameters, evaluation method and cost-sensitive decision rule.

Stage 4 - Divide the Validation Responsibilities

Student A - Analytical Validation:

- Compute the expected cost of two alternative decision thresholds using a confusion matrix and the 10:1 cost ratio
- Explain why accuracy is an inappropriate selection measure here
- Explain the strengths and limitations of the chosen technique for this data

Student B - Experimental Validation:

- Implement the final design and a simple baseline in Weka/Python on a public analogue data set (for example a diabetes-readmission or imbalanced medical data set)

- Evaluate with cross-validation using recall, precision and ROC/AUC, and total cost
- Show the effect of the imbalance-handling method (for example resampling or cost-sensitive learning)

Stage 5 - Cross-Defence

- Student A must explain: What experimental evidence supports the group's design?
- Student B must explain: How were the cost ratio and the decision threshold derived from the confusion matrix?

Stage 6 - Joint Evaluation

Together determine why the problem is difficult for standard accuracy-driven classifiers, what you sacrificed to obtain interpretable alerts, and under what conditions the design would fail. Conclude:

- What characteristics of a data set and a business requirement should a data miner examine before choosing a classifier?

Evidence Expected

- Both independent proposals, a design-comparison record, the final design, analytical and experimental validation, and a joint evaluation.

Expanded explanation: The hospital problem is a binary classification task in which the target is whether a patient will be readmitted within 30 days after discharge. Since only 6% of patients are readmitted, the data is highly imbalanced. A model that predicts every patient as not readmitted could achieve approximately 94% accuracy while failing to identify the patients who need additional care.

The cost of a false negative is ten times the cost of a false positive. Therefore, the model should not be selected using accuracy alone. The design should prioritise detecting high-risk patients while monitoring how many false alerts are generated. Recall, precision, ROC/AUC and an explicit total-cost calculation should be considered when comparing models and decision thresholds.

Before modelling, inspect missing values, inconsistent records, outliers and the distribution of each attribute. Numerical values such as age, length of stay and laboratory measurements may require suitable imputation and scaling depending on the selected algorithm. Categorical values such as diagnosis group and discharge medication need an appropriate encoding method. All preprocessing parameters must be learned from the training data to prevent information leakage.

Class imbalance can be handled using class weights, over-sampling, under-sampling or a cost-sensitive learning method. The effect of the selected technique should be compared with a simple baseline using the same validation procedure. A decision tree is a reasonable interpretable starting point because its paths can be translated into explanations that doctors can inspect. More complex models may be compared, but the final choice must balance predictive performance, interpretability and operational cost.

Threshold selection should be based on the relative costs and the confusion matrix at each candidate threshold. For each threshold, calculate false-negative cost plus false-positive cost using the stated 10:1 ratio, then compare total cost. The final report should explain why the chosen threshold is appropriate, what kinds of errors remain, and how the model could fail if patient populations, clinical practices or data quality change.

Mark Distribution

Question	Assessed Area	Marks
Q1	Decision Tree Induction	6
Q2	Bayesian Classification	6
Q3	Rule-Based and Associative Classification	7
Q4	Backpropagation	7
Q5	Support Vector Machines	7
Q6	Evaluating Classifiers and Predictors	7
Q7	Day-4 Design Challenge - Classification Solution	10
TOTAL		50

Contribution Record

Each pair must include the following table at the end of the submission.

Question	Student A - Primary Contribution	Student B - Primary Contribution	Joint Work
Q1	Entropy, information gain and manual decision tree	Weka J48 implementation and pruning comparison	Dataset preparation and interpretation of results

Question	Student A - Primary Contribution	Student B - Primary Contribution	Joint Work
Q2	Prior, conditional and posterior probability calculations	Naive Bayes in Weka and correlated-attribute investigation	Laplace correction and comparison of classifiers
Q3	IF-THEN rule extraction, coverage and accuracy	JRip/PART and associative classification analysis	Rule conflict resolution and recommendation
Q4	Forward pass, error terms and weight updates	MultilayerPerceptron training and parameter experiments	Result comparison and discussion of overfitting
Q5	Support vectors, hyperplane and margin calculation	Kernel, C-parameter and feature-scaling experiments	Plot interpretation and SVM suitability discussion
Q6	Confusion-matrix metrics and MAE, MSE, RMSE	Holdout, cross-validation, bootstrap and ROC experiments	Classifier comparison and interpretation of evaluation measures
Q7	Cost calculation for decision thresholds and analytical validation	Model implementation, baseline and experimental validation	Final design, preprocessing choices and joint evaluation

The contribution record must reflect the actual work completed. Primary contribution does not imply exclusive responsibility. Both students remain accountable for the complete assignment.

Individual Verification

Faculty may select any question, including a component for which the student was not designated as the primary contributor. Each student must be able to:

- Explain the calculations and the tool settings used
- Interpret the evaluation measures and experimental results
- Justify design decisions
- Respond to a modified data set or requirement

A student should not answer: "My partner did this section."

Recommended Faculty Verification Model (Viva)

For each student, conduct a brief 4-5 minute verification. Ask:

- Question 1 - Primary responsibility: one technical question from the component for which the student had primary responsibility.
- Question 2 - Partner's responsibility: one question from a component primarily completed by the partner.
- Question 3 - Transfer question: modify a condition and ask the student to reason about the consequence. For example: "If the readmission rate falls from 6% to 0.5%, what happens to your precision, and would you change the decision threshold?"
- "If Income were replaced by a customer ID attribute with a unique value for each tuple, what would the decision tree do and why?"
- "If the SVM attributes are not scaled and one attribute is measured in thousands, what happens to the margin?"

Conclusion

This assignment provides an opportunity to understand and apply the major techniques used in classification and prediction. Through decision tree induction, Bayesian classification, rule-based and associative classification, backpropagation, and support vector machines, the assignment explores how different algorithms learn patterns from data and make predictions.

The evaluation activities highlight the importance of selecting suitable performance measures rather than depending only on accuracy. Measures such as precision, recall, specificity, F-measure, ROC/AUC, and prediction errors help assess models from different perspectives. Cross-validation, holdout testing, and bootstrap estimation also help examine how reliably a model performs on unseen data.

The hospital readmission design challenge demonstrates that classifier selection must consider the nature of the data, class imbalance, missing values, the cost of false predictions, and the need for interpretable results. Overall, the assignment connects theoretical concepts with practical experiments using Weka or Python and encourages both students to calculate, implement, analyse, evaluate, and justify their decisions collaboratively.

References

Han, J., Kamber, M., and Pei, J. (2012). Data Mining: Concepts and Techniques (3rd ed.). Morgan Kaufmann.

Witten, I. H., Frank, E., Hall, M. A., and Pal, C. J. (2016). Data Mining: Practical Machine Learning Tools and Techniques (4th ed.). Morgan Kaufmann.

Mitchell, T. M. (1997). Machine Learning. McGraw-Hill.

Bishop, C. M. (2006). Pattern Recognition and Machine Learning. Springer.

Weka. (n.d.). Weka 3: Data Mining Software in Java. https://www.cs.waikato.ac.nz/ml/weka/

scikit-learn developers. (n.d.). scikit-learn User Guide. https://scikit-learn.org/stable/user_guide.html